

DESIGNARC

Furniture Manufacturing Workflow Platform

SOLUTION ARCHITECTURE & DESIGN BLUEPRINT

A 16-Phase Enterprise Architecture Engagement — from Business Analysis to Delivery

Field	Value
Prepared for	DesignArc — Interior Designers & Consultants
Basis document	Software Requirements Specification, Rev 1.1 (18 Jul 2026, ISO/IEC/IEEE 29148:2018)
Document type	Solution Architecture Document (SAD) + Design Blueprint
Status	Draft for stakeholder review
Date	21 July 2026
Scope	MVP → Version 1.0, with a forward roadmap to v2.0 / v3.0

Confidential — prepared as an independent architecture response to the DesignArc SRS. Every recommendation in this document states the reasoning behind it and, where a meaningful alternative exists, why the chosen option is preferred.

Contents

Contents.....	2
Executive Summary.....	5
Headline recommendations	5
Phase 1 — Business Analysis.....	6
1.1 The core business problem	6
1.2 The existing (as-is) process	6
1.3 Pain points (ranked by business impact)	6
1.4 Users and stakeholders.....	6
1.5 Business goals and KPIs.....	7
1.6 Risks (with mitigations).....	7
1.7 Assumptions and constraints.....	7
1.8 Challenges to the requirements (what we would push back on)	8
1.9 Missing information / open questions.....	8
Phase 2 — Requirement Analysis	10
2.1 Functional requirements (MoSCoW)	10
2.2 Hidden / implicit requirements the SRS should make explicit.....	10
2.3 Business rules (canonical list)	11
2.4 Validation rules	11
2.5 Edge cases & error cases.....	11
2.6 Non-functional requirements	12
2.7 Security & compliance requirements	12
2.8 Out of scope (confirmed) and future requirements	12
Phase 3 — Solution Architecture	13
3.1 System overview & context	13
3.2 Architecture style — modular monolith (and why not microservices).....	13
3.3 Technology stack (with rationale).....	14
3.4 Cross-cutting architectural decisions.....	15
Phase 4 — Database Design	17
4.1 Entities & the ER model	17
4.2 Core tables	17
4.3 Keys, relationships & integrity	18
4.4 Indexes	18
4.5 Master, reference, audit, history & soft-delete strategy.....	18
4.6 Data integrity rules (enforced in the schema)	19
Phase 5 — Feature Planning (Modules).....	20

Phase 6 — User Roles	23
6.1 Role responsibilities & restrictions	23
6.2 Permission matrix (RBAC contract)	23
6.3 User journeys	23
Phase 7 — API Planning	25
7.1 Conventions	25
7.2 Representative endpoints	25
7.3 Example — set price (success & errors)	25
Phase 8 — UI Planning	27
8.1 Design system foundations	27
8.2 Admin web screens	27
8.3 Mobile screens (field roles)	27
8.4 Navigation flow	27
Phase 9 — Business Process	29
9.1 Current vs. future process	29
9.2 End-to-end workflow	29
9.3 Stage state machine	31
9.4 Post-inspection pricing sequence	31
9.5 Decision points, notifications & escalations	32
Phase 10 — Security	33
10.1 Authentication & session security	33
10.2 Authorization (RBAC)	33
10.3 OWASP Top-10 mitigations	33
10.4 Rate limiting, secrets, audit	33
10.5 Compliance	34
Phase 11 — Performance & Scalability	35
11.1 Capacity estimate	35
11.2 Performance targets & how we meet them	35
11.3 Caching, load balancing & CDN	35
Phase 12 — DevOps	36
12.1 Git & branching	36
12.2 Environments & CI/CD	36
12.3 Containers, secrets, backups, monitoring	36
Phase 13 — Testing Strategy	37
13.1 Automation strategy	37
Phase 14 — Documentation	38

Phase 15 — Project Planning	39
15.1 Epics & feature breakdown.....	39
15.2 Indicative sprint plan (2-week sprints)	39
15.3 Team structure.....	39
15.4 Milestones, dependencies & effort	40
15.5 Risk register (delivery)	40
Phase 16 — Improvements Beyond the Requirements.....	41
16.1 AI, ML & automation.....	41
16.2 Analytics & enterprise features	41
16.3 UX, notifications & integrations.....	41
16.4 Cost & security optimisations	41

Executive Summary

DesignArc runs a furniture-manufacturing and interior-design business in which work flows from a project, into job cards, through a carpentry stage and then a painting stage, each physically inspected by a supervisor before a price is agreed and the worker is paid. Today that flow is coordinated by paper, phone calls and verbal handoffs. The SRS (Rev 1.1) captures the target digital workflow well: it is testable, traceable and ISO/IEC/IEEE 29148-conformant. This document is our architectural answer to it.

The defining characteristic of this system is not technical complexity — the data volumes are modest and the user base small — but workflow integrity. The business explicitly names inspection sign-off, price setting/acceptance and payment records as business-critical, and requires that pricing history, inspection photos and earnings can never be lost. That single fact drives almost every architectural decision here: a strongly-consistent relational core, an append-only pricing ledger, a rigorously enforced stage state machine, and RBAC on every endpoint. We optimise for correctness and auditability first, then for the modest scale and cost the business actually needs.

Headline recommendations

- **Architecture:** a modular monolith (single deployable, cleanly separated modules) behind an API gateway — not microservices. At 200 concurrent users and one tenant, microservices would add operational cost and distributed-transaction risk with zero business upside. The module boundaries are drawn so that a future service (e.g. Notifications, Reporting) can be extracted if scale ever demands it.
- **Stack:** Flutter for the single cross-platform mobile app, React + TypeScript for the Admin web SPA, NestJS (Node 24 LTS) for the API, PostgreSQL 18 as the system of record, Redis for cache/queue, and cloud object storage with signed URLs for design files and inspection photos.
- **Workflow engine:** the nine-state stage lifecycle is implemented as an explicit, server-enforced state machine with guarded transitions, so an illegal move (e.g. pricing before approval, or painting before carpentry approval) is impossible by construction rather than by convention.
- **Pricing ledger:** all pricing events are written to an append-only, immutable history table; the ‘accepted price’ is a projection of that ledger, giving DesignArc a defensible audit trail for every rupee a worker is paid.
- **Delivery:** an estimated 5–6 month MVP-to-v1.0 programme delivered by a small cross-functional team of 5–6, through a containerised CI/CD pipeline across dev, staging, UAT and production.

We also challenge several requirements below (single-supervisor bottleneck, ‘reliable internet’ assumption, currency handling, deletion semantics) and recommend concrete resolutions for every open TBD in the SRS.

Phase 1 — Business Analysis

Understand the business before designing the software. This phase establishes the problem, the people, the goals and — crucially — the hidden assumptions and risks the SRS leaves implicit.

1.1 The core business problem

DesignArc coordinates skilled manual work (carpentry, then painting) across a workshop floor, and pays workers per completed, inspected and approved item. The business problem is not ‘we need an app’; it is that the coordination and money-settlement layer is invisible and unaccountable. Specifically:

- Work is assigned verbally, so there is no reliable record of who is doing what, or since when.
- Quality sign-off lives in a supervisor's head and memory, with no photographic proof, so disputes about whether work was acceptable cannot be settled objectively.
- Pricing is negotiated ad hoc and rarely written down, so neither side has a trustworthy history of what was proposed, revised, accepted or declined.
- Payments are reconciled manually from that same unreliable memory, which is slow, error-prone, and erodes worker trust.

The one-line framing: DesignArc needs a single source of truth for the lifecycle of every piece of work — from assignment to inspection to agreed price to payment — that is transparent to both management and workers.

1.2 The existing (as-is) process

Step	How it works today	Weakness
Assign work	Admin/supervisor tells a carpenter verbally or by phone	No record; disputes over what was asked
Do the work	Worker completes item on the floor	No visibility of progress
Inspect	Supervisor eyeballs the item	No evidence captured; subjective
Price	Price discussed informally, often after the fact	No history; renegotiation is opaque
Pay	Amounts tallied from memory/notebook	Errors; delays; low worker trust
Report	No consolidated view exists	Management flies blind

1.3 Pain points (ranked by business impact)

#	Pain point	Who feels it	Impact
1	Payment disputes from unrecorded prices	Workers, Admin	High
2	No proof of work quality	Supervisor, Admin	High
3	Lost/forgotten assignments	Whole floor	High
4	No real-time view of project progress	Admin	Medium
5	Slow, manual payment reconciliation	Admin	Medium
6	Worker distrust from opaque pricing	Workers	Medium

1.4 Users and stakeholders

The SRS names four operational roles. It is worth separating the people who use the system daily (users) from those with an interest in its success (stakeholders), because their needs differ.

Party	Type	Primary interest
Admin	User (web)	Control, visibility, accurate payments, low admin overhead
Carpenter	User (mobile)	Fair, transparent pay; simple task flow; proof they did the work
Painter	User (mobile)	Same as carpenter, for the painting stage
Supervisor	User (mobile)	Fast inspection capture; defensible sign-off
Business owner	Stakeholder	ROI, reduced disputes, scalable operation
QueMore delivery/QA	Stakeholder	Buildable, testable, maintainable system
Future: Finance role	Stakeholder	Separation of payment authority (deferred v2.0)

1.5 Business goals and KPIs

We translate the SRS ‘business benefits’ into measurable goals, because an unmeasurable goal cannot be managed.

Business goal	KPI	Target
Eliminate verbal handoffs	% of assignments made in-system	100% within 1 month of go-live
Transparent pricing	% of paid stages with full pricing history	100%
Proof of work	% of approved stages with ≥ 1 photo	100% (enforced)
Faster settlement	Median days from Completed $\rightarrow$ Paid	$\downarrow$ vs. baseline; target < 7 days
Reduce disputes	# of payment disputes / month	$\downarrow$ 80% within 6 months
Real-time visibility	Admin dashboard adoption	Daily active use by Admin

1.6 Risks (with mitigations)

Risk	Likelihood	Impact	Mitigation
Single-supervisor bottleneck stalls the whole floor	Medium	High	Design role model to allow N supervisors even in MVP; see §1.8 challenge C1
Poor workshop connectivity delays photo upload	Medium	Medium	Resumable/queued uploads; graceful retry; compress on device
Loss of pricing/photo/earning data	Low	Very High	Append-only ledger, PITR backups, transactional writes, restore drills
Worker device fragmentation / low-end phones	Medium	Medium	Flutter + lightweight UI; test on low-end Android
Scope creep toward payment disbursement	Medium	High	Hold the line: record-only; disbursement is v2.0+ (ASM-05)
Currency/locale ambiguity (LKR vs multi)	High	Low	Resolve TBD-02 now; default to LKR single-currency (see Phase 2)

1.7 Assumptions and constraints

Assumptions carried from the SRS (validated)

- All field workers have a capable smartphone (ASM-01) — if false, a paper fallback is required and the value case collapses.

- Each worker has a unique account keyed to email or phone (ASM-02) — essential for attributing pay.
- Reasonable internet exists at the workshop for uploads (ASM-03) — we harden against this being intermittent rather than assuming it away.
- One supervisor inspects both stages (ASM-04) — we treat this as a configuration default, not a hard architectural limit (see challenge C1).
- The system records payments but never disburses money (ASM-05) — a firm scope boundary that keeps DesignArc out of payments-regulation territory.

Constraints

- **Regulatory:** personal data (identities, payment records) must support correction and erasure where no legal retention rule applies.
- **Security:** bcrypt (cost ≥ 12), TLS 1.2+, signed JWT, RBAC on every endpoint, access-controlled media.
- **Platform:** current Android/iOS; current Chrome, Edge, Firefox, Safari.
- **Brand:** forest green #4E7A54 primary, brick red #B23A2E accent, cream #F7F4EE background.
- **Deployment:** single-tenant, cloud-hosted, containerised.

1.8 Challenges to the requirements (what we would push back on)

A good architecture partner does not just build what is written; it flags where the written requirement will cause pain. Five points:

ID	Challenge	Recommendation
C1	'One Supervisor' (ASM-04) creates a single point of failure and a floor-wide bottleneck; it also weakens separation of duties if that person is unavailable.	Model Supervisor as a role that many users can hold. Keep 'one on duty' as an operational choice, not a schema limit. Zero extra MVP cost, large future saving.
C2	'Reliable internet' (ASM-03) is optimistic for a workshop floor; the SRS then rules out offline-first.	Adopt a pragmatic middle: read-mostly caching + resumable background uploads so a dropped connection delays but never loses an inspection.
C3	Currency/locale is left as TBD-02 but pricing/earnings are core.	Decide now: single-currency LKR, stored as integer minor units. Multi-currency is a costly generalisation with no current business need.
C4	Deletion semantics are only partly specified.	Make soft-delete the default for Users and Projects; make Pricing History, Photos and Earnings strictly immutable/retained. Hard delete only via a governed data-erasure process for GDPR-style requests.
C5	Admin holds both pricing authority and payment-marking authority — a separation-of-duties weakness.	Acceptable for MVP given team size, but design permissions so a Finance role can be split out in v2.0 without refactoring (TBD-03).

1.9 Missing information / open questions

These map to the SRS TBD log; we recommend defaults in Phase 2 so the build is not blocked, but they should be confirmed with DesignArc.

- TBD-01 — confirm final stack & hosting provider (we recommend one below).
- TBD-02 — currency/locale (recommend LKR-only).

- TBD-03 — separate Finance role for payments (recommend deferring to v2.0, design-ready now).
- TBD-04 — retention period for photos/records (recommend 7 years for financial records, aligned to typical tax retention; confirm with DesignArc/legal).
- TBD-05 — max photo count/size per inspection (recommend ≤ 10 photos, ≤ 5 MB each, matching NFR-P.5).

Phase 2 — Requirement Analysis

Consolidate the SRS into a prioritised, testable requirement set, adding the hidden requirements, edge cases and validation rules the SRS does not yet make explicit. Prioritised with MoSCoW.

2.1 Functional requirements (MoSCoW)

The SRS FR-1...FR-9 groups are sound. Below we restate them by module with a MoSCoW priority and note additions (marked +) that experience says will be needed.

Ref	Requirement	MoSCoW
FR-1.x	Authentication, RBAC (4 roles), user CRUD, password reset, block+log unauthorised actions	Must
FR-2.x	Project create/edit/archive, unique ID, completion roll-up, protect projects with payments	Must
FR-3.x	Job cards, stage model (Carpentry/Painting), sequence gate, 9-state status machine, media store	Must
FR-4.x	Stage assignment to role-matched worker, no price at assignment, notify assignee	Must
FR-5.x	Mark ready, supervisor photo upload, approve/reject with reason, require ≥ 1 photo	Must
FR-6.x	Post-inspection pricing: set $\rightarrow$ accept/decline $\rightarrow$ revise, immutable pricing history	Must
FR-8.x	Earning on accept, Paid/Unpaid, mark paid, guard earnings before acceptance	Must
FR-7.x	In-app notifications on every handoff; unread indicator	Should
FR-7.3	Push notifications to mobile	Should
FR-9.1/9.2	Admin dashboard KPIs + action queues	Should
FR-9.3/9.4	Monthly worker report + PDF export	Should
FR-9.5	Worker workload view for Admin	Could
+FR-A	Audit trail viewer for Admin (read the security audit log)	Could
+FR-B	Bulk assignment / re-assignment of stages	Won't (MVP)

2.2 Hidden / implicit requirements the SRS should make explicit

These are behaviours the system will need but that are not spelled out. Surfacing them now prevents rework.

- Re-assignment: what happens if a worker leaves mid-stage? Need an Admin re-assign action that preserves history and re-notifies.
- Concurrent action safety: two Admins setting a price, or a worker accepting while an Admin revises — needs optimistic locking on the stage.
- Idempotency: a flaky mobile connection may resend 'accept price'; the server must treat repeats as one action.
- Notification fan-out on rework: rejecting a stage must notify the worker and update the Admin dashboard counts.
- Time zone / clock: all timestamps ISO 8601 in UTC, displayed in DesignArc's local time (Asia/Colombo).
- Photo integrity: store original + generate a thumbnail; never allow a stage to be approved with zero photos (already FR-5.6, reinforce at DB level).

- Account deactivation vs. open work: deactivating a worker with in-flight stages must be handled (reassign or block deactivation).

2.3 Business rules (canonical list)

Rule	Statement
BR-1	Each account has exactly one active primary role; only Admin creates/deactivates accounts.
BR-2	Only Admin creates/edits/archives Projects; a Project with recorded payments can be archived, never deleted.
BR-3	No price exists on a stage before Supervisor Approval; Painting cannot start before Carpentry is Approved on the same job card.
BR-4	A stage is assigned to exactly one worker whose role matches the stage type; no price at assignment.
BR-5	Only the Supervisor uploads inspection photos and approves/rejects; rejection returns the same stage to the same worker.
BR-6	Only Admin sets/revises price; only the assigned worker accepts/declines; the accepted price is the basis for payment.
BR-7	A notification is addressed only to the role(s) party to the triggering event.
BR-8	An earning exists only for a stage whose price is Accepted; only Admin marks it Paid.
BR-9	Pricing history is append-only; entries are never modified or deleted.

2.4 Validation rules

Field / action	Rule
Email	RFC-5322 valid, unique, case-insensitive; or phone E.164 if used as identifier
Password	≥10 chars, mixed classes; checked against a breached-password list; bcrypt cost ≥12
Price	> 0, integer minor units (LKR cents), ≤ a sane ceiling; two-decimal display
Photos on approve	≥1, ≤10, each ≤5 MB, JPEG/PNG/HEIC; server re-validates type by magic bytes
Reject reason	Mandatory, 5–500 chars
Project name	1–120 chars, unique per client is recommended (warn, not block)
Stage transition	Only transitions permitted by the state machine (Phase 4) are accepted

2.5 Edge cases & error cases

Scenario	Expected handling
Worker declines price repeatedly	Allowed; every round appended to history; Admin may keep revising; no auto-resolution
Supervisor approves then a photo upload fails	Approval is transactional with ≥1 successfully stored photo; partial upload = no approval
Two Admins set price simultaneously	Optimistic lock; second write rejected with a conflict message
Worker accepts an already-revised price	Version check; stale acceptance rejected, worker re-prompted with current price
Deactivate worker with in-progress stage	Blocked with guidance to reassign the stage first
Delete project with payments	Blocked; archive offered instead (FR-2.5)
Notification service down	Workflow proceeds; notification retried from queue; never blocks the state change
Duplicate 'mark ready' tap offline	Idempotent; single state change recorded

2.6 Non-functional requirements

Category	Target (from SRS, refined)
Performance	Primary screens < 3s p95 on 4G; price actions < 2s p95; photo upload ($\leq 10 \times 5$ MB) < 15s p95; report < 5s p95
Scalability	200 concurrent users with < 20% degradation; horizontally scalable stateless API
Availability	99.5% monthly uptime excl. maintenance; health checks + alerting
Reliability	No loss of pricing/photo/earning data once committed; transactional rollback on partial failure
Security	OWASP ASVS-aligned; RBAC everywhere; audit log of auth failures & permission denials
Usability	Accept/decline within 3 taps; first inspection in < 3 min unaided; WCAG 2.1 AA on key journeys
Maintainability	Modular architecture, documented API, $\geq 80\%$ coverage on business logic, CI/CD
Portability	Current Android/iOS + evergreen browsers; container-deployable backend

2.7 Security & compliance requirements

- Authenticate every request with a signed JWT; authorise every action via RBAC.
- Encrypt in transit (TLS 1.2+) and at rest (DB + object storage encryption).
- Audit authentication failures and permission-denied events with user, timestamp, action.
- Support correction and erasure of personal data where no legal retention rule applies (right to rectification/erasure).
- Follow OWASP ASVS for auth, access control and data protection; timestamps in ISO 8601.

2.8 Out of scope (confirmed) and future requirements

Out of scope (MVP/v1.0)	Future (roadmap)
Online payment disbursement / payroll / bank integration	Disbursement & payroll (v2.0+, re-scopes regulatory posture)
Materials / stock / inventory	Materials & inventory tracking (v3.0)
Client-facing portal	Client portal (v3.0)
External accounting integration	Accounting integration (v3.0)
Multi-language UI, full offline-first	Separate Finance role & analytics (v2.0); i18n if expansion demands

Phase 3 — Solution Architecture

The overall shape of the system, the architecture style and every technology choice — each with the reasoning behind it and the alternative it beats.

3.1 System overview & context

DesignArc is a new, self-contained platform: a responsive Admin web application and a cross-platform mobile app for field roles, served by one shared REST API over a relational system of record, with cloud object storage for files and a push service for mobile alerts. It integrates with almost nothing external by design — that narrow boundary is a feature, because it keeps the security surface and the operational burden small.

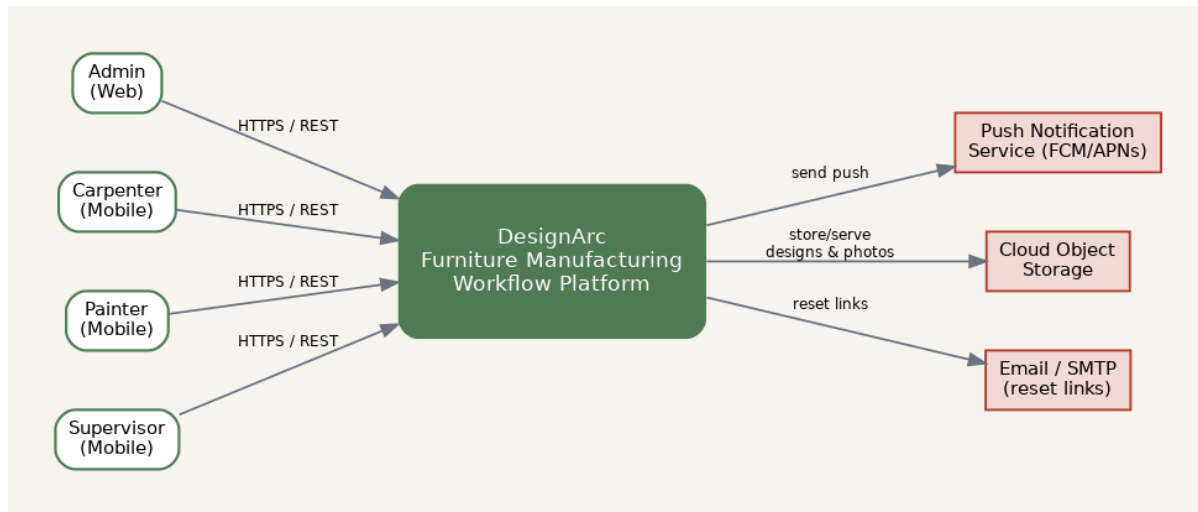


Figure 1 — System context: four roles, one platform, three external services.

3.2 Architecture style — modular monolith (and why not microservices)

We recommend a modular monolith: a single deployable application internally divided into strongly-bounded modules (Auth, Projects, Job Cards & Stages, Pricing, Payments, Notifications, Reporting), plus a small set of asynchronous background workers. Here is the reasoning, weighed against the obvious alternative.

Criterion	Modular monolith (recommended)	Microservices (rejected for now)
Scale fit	Ideal at 200 concurrent users / single tenant	Built for scale this system will not reach
Transaction integrity	One DB, ACID transactions across pricing→earning in a single commit	Distributed transactions / sagas needed — real risk to the money path
Operational cost	One pipeline, one deploy, one DB to run	N services, service mesh, distributed tracing — heavy for a small team
Dev speed	Fastest to MVP; refactor across modules is trivial	Slow; cross-service change is expensive
Future expansion	Module seams allow extracting a service later if needed	N/A

Why this is the better decision: the business-critical requirement is transactional integrity of the inspection→price→earning path. A monolith gives that for free in a single ACID transaction; microservices would force us to coordinate that across services and re-introduce exactly the ‘partial record’ risk the SRS

forbids. We keep clean module boundaries so that if Notifications or Reporting ever needs independent scaling, it can be lifted out — but we do not pay the distributed-systems tax before there is a reason to.

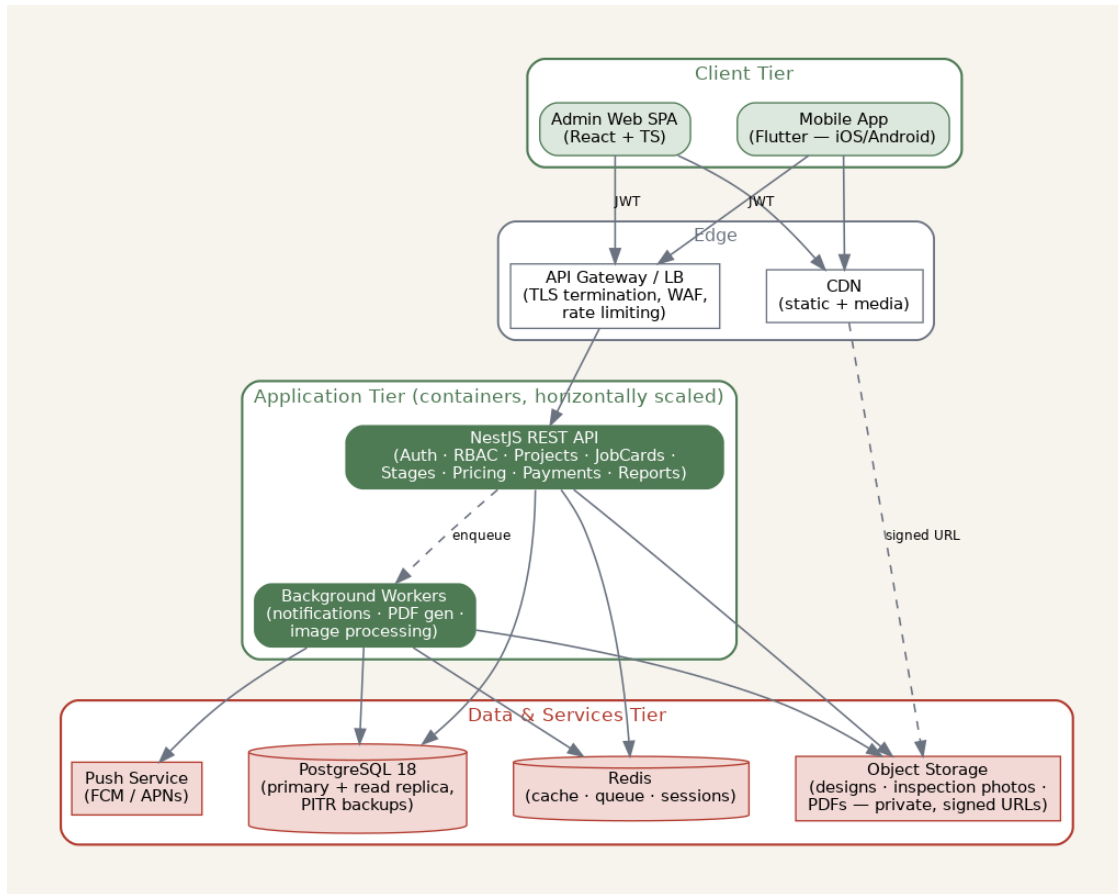


Figure 2 — Logical & deployment architecture (client, edge, application, data tiers).

3.3 Technology stack (with rationale)

Each choice below optimises the SRS priorities: correctness first, then cost, dev speed, UX and future expansion. Versions are current stable/LTS as of July 2026.

Layer	Recommendation	Why this, over the alternative
Mobile	Flutter 3.41 (Dart), single codebase iOS+Android	One codebase for both field platforms halves build/maintenance cost; excellent camera + image handling for inspection photos; smooth on low-end Android. Chosen over React Native for consistent rendering and a single UI toolkit; chosen over native (2× the work) for cost/speed.
Admin web	React 18 + TypeScript + Vite	Rich, data-dense dashboard needs a mature component ecosystem; TS gives compile-time safety on the money-handling UI. Chosen over server-rendered pages because the Admin app is an interactive SPA, not content.
Backend	NestJS on Node 24 LTS (TypeScript)	Opinionated, modular structure enforces the module boundaries we want; DI + guards make RBAC and the state machine clean and testable; shared TS types with the web app. Chosen over bare Express (too unstructured) and over a second language (one language = one hiring pool, shared DTOs).

Layer	Recommendation	Why this, over the alternative
Database	PostgreSQL 18	ACID guarantees for the money path; rich constraints (CHECK, FK, partial indexes) enforce business rules in the data itself; JSONB for flexible notification payloads; proven, low-cost, portable. Chosen over MongoDB because this domain is deeply relational and integrity-critical.
Cache / queue	Redis 7	Sub-ms cache for dashboards; reliable background-job queue (BullMQ) for notifications, PDF and image processing. Chosen over a DB-based queue for throughput and simplicity.
Auth	JWT access + refresh tokens; bcrypt (cost ≥ 12)	Stateless access tokens scale horizontally; short-lived access + rotating refresh limits blast radius. Meets SRS security constraints exactly.
File storage	Cloud object storage (S3-compatible), private buckets, signed URLs	Cheap, durable, scalable for designs/photos; media served only to authorised users via short-lived signed URLs. Chosen over storing blobs in the DB (bloats backups, slow).
Push	FCM (Android) + APNs (iOS) via one abstraction	Native, free, reliable mobile delivery. Abstraction layer lets us swap providers.
PDF	Server-side HTML→PDF (Playwright/Chromium) in a worker	Pixel-accurate branded monthly reports; generated off the request path.
Cloud	One mainstream provider (AWS/Azure/GCP) — managed Postgres, object store, container runtime	Managed services shrink ops load for a small team; single-provider keeps billing and IAM simple.
Containers / orchestration	Docker; start on a managed container service, Kubernetes-ready	Right-sized ops: no k8s complexity on day one, but images are portable if scale demands it.
CI/CD	GitHub Actions (or GitLab CI)	Build→test→scan→deploy on every merge; enforces the $\geq 80\%$ coverage gate.
Observability	Centralised logging + metrics + alerting (e.g. OpenTelemetry → managed backend)	Meets the 99.5% uptime & health-check NFRs; structured logs feed the audit trail.
Search / analytics	Postgres full-text + SQL aggregates (MVP)	No dedicated search engine needed at this scale; revisit only if data grows.

3.4 Cross-cutting architectural decisions

- **Stateless API:** no server-side session state (JWT-based), so any instance can serve any request — this is what makes horizontal scaling to the 200-user target trivial.
- **State machine as a first-class component:** stage transitions run through a single guarded service, so business rules BR-3/5/6 are enforced in exactly one place.
- **Append-only pricing ledger:** writes only; the ‘current price’ is derived. This makes the audit trail tamper-evident and satisfies FR-6.6 / BR-9 structurally.
- **Async by default for non-critical work:** notifications, push, PDF and thumbnailing run on the queue so a slow third party never blocks a state change (satisfies the SRS ‘notifications are non-critical’ rule).
- **Signed-URL media access:** clients get time-limited URLs; the API authorises, the storage serves — no large files pass through the app tier.

Phase 4 — Database Design

A relational schema that encodes the business rules as constraints, keeps money and evidence immutable, and is fully auditable.

4.1 Entities & the ER model

The SRS logical model is well-formed. We refine it into an implementable schema: Attachment is split from generic media so design files and inspection photos are cleanly separated, Earning carries who/when it was paid, and every table gets audit columns.

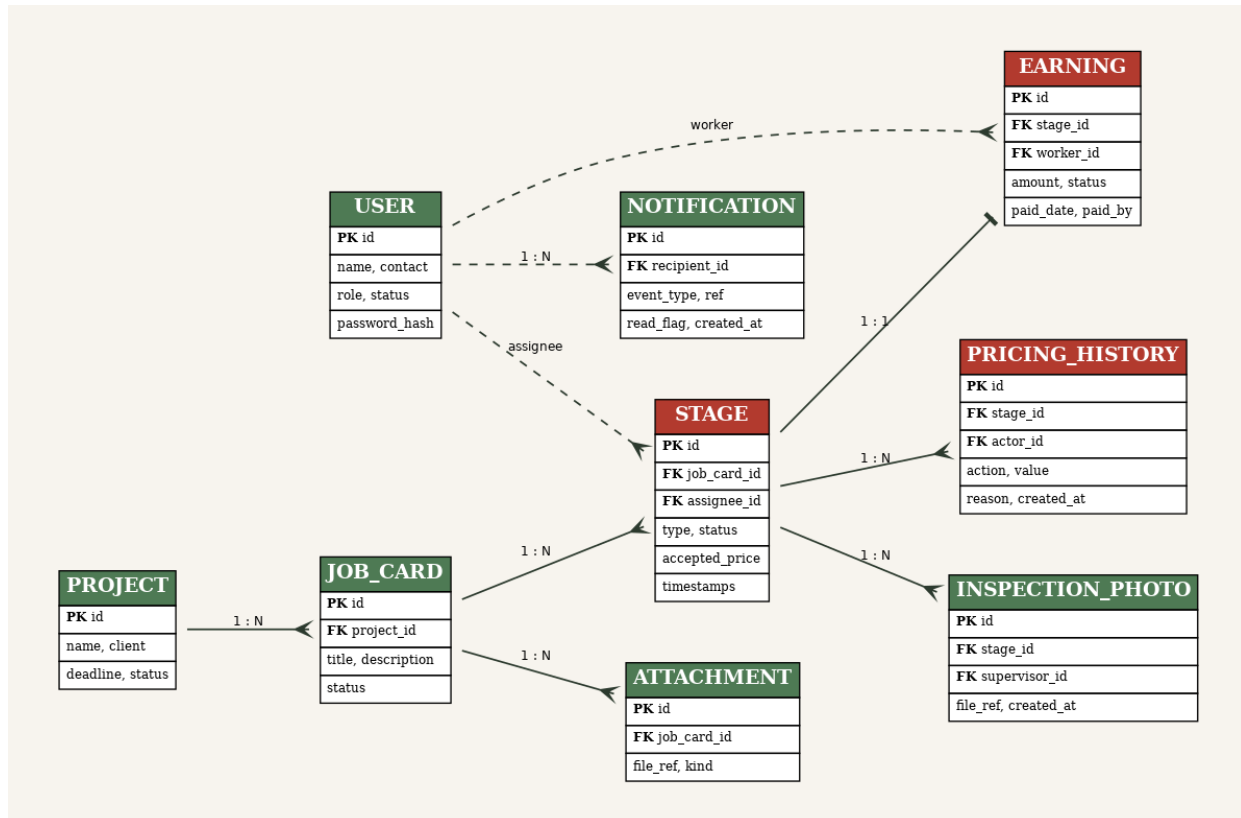


Figure 3 — Entity–relationship diagram (PK/FK highlighted; red = money/evidence entities).

4.2 Core tables

Entity	Key attributes	Notes
user	id (PK), name, email/phone (unique), role, status, password_hash, created_at, updated_at, deleted_at	One active role; soft-delete via status + deleted_at
project	id (PK), name, client, description, deadline, status, created_by, timestamps	Archived not deleted when it has payments
job_card	id (PK), project_id (FK), title, description, status, timestamps	Belongs to a project; has stages + attachments
attachment	id (PK), job_card_id (FK), file_ref, kind (design), uploaded_by, created_at	Design files for the brief

Entity	Key attributes	Notes
stage	id (PK), job_card_id (FK), type (carpentry painting), sequence_no, assignee_id (FK), status, accepted_price, version, timestamps	Heart of the workflow; version column for optimistic locking
pricing_history	id (PK), stage_id (FK), actor_id (FK), action (proposed revised accepted declined), value, reason, created_at	Append-only ledger — INSERT only
inspection_photo	id (PK), stage_id (FK), supervisor_id (FK), file_ref, thumb_ref, created_at	≥1 required to approve
earning	id (PK), stage_id (FK, unique), worker_id (FK), amount, status (unpaid paid), paid_date, paid_by, created_at	1:1 with a Completed stage
notification	id (PK), recipient_id (FK), event_type, ref_type, ref_id, read_flag, created_at	Per-user feed
audit_log	id (PK), actor_id, action, entity, entity_id, meta (JSONB), ip, created_at	Auth failures, permission denials, sensitive actions

4.3 Keys, relationships & integrity

Aspect	Decision & why
Primary keys	UUID v7 (time-ordered) surrogate PKs — globally unique, safe to expose, index-friendly, no guessable sequential IDs
Foreign keys	Enforced at DB level on every relationship; ON DELETE RESTRICT for money/evidence, so history can never be orphaned
Uniqueness	user.email/phone unique; earning.stage_id unique (guarantees BR-8 one-earning-per-stage)
Sequence gate	Partial constraint / service check: a painting stage cannot leave 'Assigned' until the sibling carpentry stage is Approved (BR-3)
Price guard	CHECK: accepted_price is NULL unless status ∈ {price accepted, completed}; enforces 'no price before approval'
Concurrency	stage.version (optimistic lock) prevents lost updates on simultaneous price/accept actions

4.4 Indexes

Index	Purpose
stage(status)	Dashboard counts by status; action queues
stage(assignee_id, status)	Worker workload & 'my tasks' screens
stage(job_card_id, sequence_no)	Stage tracker ordering
pricing_history(stage_id, created_at)	Fast chronological price history
earning(worker_id, status, created_at)	Monthly report per worker (BR-9.2)
notification(recipient_id, read_flag)	Unread indicator & feed
project(status)	Active-project filtering

4.5 Master, reference, audit, history & soft-delete strategy

Concern	Approach	Why
Reference/master data	role, stage_type, stage_status, event_type as enums/lookup tables	Constrains values; makes reports consistent; cheap to extend
Audit tables	audit_log (security + sensitive actions); DB-level created_by/updated_by everywhere	Meets SRS audit requirement; supports dispute resolution

Concern	Approach	Why
History tables	pricing_history is the canonical append-only history; earnings never mutated after paid	Tamper-evident money trail (BR-9)
Soft delete	Users & Projects: status + deleted_at, filtered out of active views; never hard-deleted where related records exist	Preserves referential history; supports FR-2.5
Hard delete / erasure	Only via a governed GDPR-style erasure job that anonymises PII while retaining financial records required by law	Balances erasure right vs retention duty
Retention	Financial records & photos retained per TBD-04 (recommend 7 years); enforced by retention policy, not ad-hoc deletion	Legal defensibility

4.6 Data integrity rules (enforced in the schema)

- Every Project, Job Card, Stage, Payment has a system-generated UUID (never user-supplied).
- pricing_history is INSERT-only; UPDATE/DELETE are revoked at the role level in Postgres.
- An earning row can exist only for a stage whose status is Completed (enforced by trigger + unique FK).
- Users and Projects are soft-deleted where related records exist; hard delete is blocked by FK RESTRICT.
- All timestamps stored in UTC (timestampz), rendered in Asia/Colombo for users.

Phase 5 — Feature Planning (Modules)

The system decomposed into eight modules. Each entry gives purpose, features, workflow, permissions, validations, dependencies and future enhancements.

M1 — Identity & Access

Aspect	Detail
Purpose	Control who enters the system and what each role may do
Features	Login, JWT issue/refresh, password reset, Admin user CRUD, role assignment, deactivate
Sub-features	Breached-password check, failed-login lockout, RBAC guard on every route, permission-denied audit
Workflow	Admin creates account → user notified → user sets password → logs in → role-scoped home
Permissions	Only Admin manages accounts; all roles manage own password
Validations	Unique email/phone; password policy; exactly one active role
Dependencies	Notifications (welcome/reset), Audit
Future	SSO/2FA; separate Finance role (v2.0)

M2 — Project Management

Aspect	Detail
Purpose	Group related job cards for one client engagement
Features	Create/edit/archive project, unique ID, completion roll-up, protect projects with payments
Workflow	Admin creates project → adds job cards → completion % updates as stages are Approved
Permissions	Admin only for write; all internal roles may view relevant projects
Validations	Name length; cannot delete when payments exist (archive instead)
Dependencies	Job Cards & Stages
Future	Deadlines/overdue alerts (v2.0), portfolio analytics

M3 — Job Cards & Stages

Aspect	Detail
Purpose	Model a unit of furniture work and its sequential stages
Features	Create job card + design attachments (no price), configure stages, 9-state status machine, sequence gate, media store
Sub-features	Stage tracker, attachment viewer, per-stage timeline
Workflow	Admin creates job card → configures Carpentry (+optional Painting) → stages progress via the state machine
Permissions	Admin creates/configures; worker/supervisor act on assigned stages
Validations	≥1 attachment; painting locked until carpentry Approved (BR-3)
Dependencies	Assignment, Inspection, Pricing
Future	More stage types (e.g. upholstery), templates

M4 — Assignment

Aspect	Detail
Purpose	Put the right worker on each stage with the design brief, no price
Features	Assign/reassign stage to role-matched worker, notify assignee, expose designs
Workflow	Admin assigns → worker notified → worker opens brief → starts work
Permissions	Admin assigns; assigned worker views
Validations	Role must match stage type; no price at assignment (BR-4)
Dependencies	Identity, Job Cards, Notifications
Future	Skills-based/auto suggestion, bulk assign

M5 — Work Execution & Inspection

Aspect	Detail
Purpose	Capture work completion and independent supervisor sign-off with evidence
Features	Mark ready, supervisor photo upload, approve/reject with reason, require ≥1 photo
Workflow	Worker marks ready → supervisor notified → inspects, uploads photos → approve (→pricing) or reject (→rework)
Permissions	Only supervisor uploads/approves/rejects; only assignee marks ready
Validations	≥1 photo to approve; mandatory reject reason; transactional approval
Dependencies	Media storage, Pricing (approval triggers it), Notifications
Future	Photo annotation, checklist-based inspection

M6 — Post-Inspection Pricing

Aspect	Detail
Purpose	Agree the price after approval, with a full immutable history
Features	Enable-on-approve, Admin sets price, worker accept/decline, Admin revise, append-only history
Workflow	Approved → Admin sets price (Proposed) → worker accepts (Completed→earning) or declines → Admin revises → repeat
Permissions	Only Admin sets/revises; only assigned worker accepts/declines (BR-6)
Validations	Price>0; pricing only after approval (BR-3); version check on accept
Dependencies	Inspection, Payments (creates earning), Notifications
Future	Rate cards / suggested pricing, bulk pricing

M7 — Payments & Financial Records

Aspect	Detail
Purpose	Record what each worker earned and whether it is paid (record-only)
Features	Auto earning on accept, Paid/Unpaid status, mark paid (date+admin), guard against premature earnings
Workflow	Price accepted → earning (Unpaid) → Admin marks Paid → recorded
Permissions	Only Admin marks paid (BR-8)
Validations	Earning only for Completed stage; immutable amount
Dependencies	Pricing, Reporting
Future	Disbursement & payroll integration (v2.0+)

M8 — Notifications, Dashboard & Reporting

Aspect	Detail
Purpose	Keep everyone informed and give management/workers visibility
Features	In-app + push on every handoff, unread indicator, Admin KPI dashboard, action queues, monthly worker report + PDF, workload view
Workflow	Event → queued notification → in-app + push; Admin sees live KPIs; worker exports monthly PDF
Permissions	Recipients see only their notifications; worker sees only own report (BR-9.1)
Validations	Report includes only stages Completed within the month (BR-9.2)
Dependencies	All modules (event sources), Redis queue, PDF worker
Future	Overdue/deadline alerts, richer analytics, exportable dashboards

Phase 6 — User Roles

Four roles, each with responsibilities, permissions, restrictions and a journey. The permission matrix is the contract the RBAC layer enforces.

6.1 Role responsibilities & restrictions

Role	Responsibilities	Restrictions
Admin	Create projects/job cards, assign stages, set/revise prices, mark payments, monitor dashboard	Cannot upload inspection photos or approve work (separation from inspection)
Carpenter	Execute carpentry stages, mark ready, accept/decline price, view own earnings/report	Cannot see other workers' pay; cannot self-approve or self-price
Painter	Execute painting stages (after carpentry approved), same flow as carpenter	Same as carpenter; cannot start before carpentry approved
Supervisor	Inspect finished stages, upload photos, approve/reject	Cannot set price, assign, or mark payment

6.2 Permission matrix (RBAC contract)

Action	Admin	Carpenter	Painter	Supervisor
Manage users	✓	—	—	—
Create/edit project	✓	—	—	—
Create job card / configure stages	✓	—	—	—
Assign / reassign stage	✓	—	—	—
Mark stage ready	—	own	own	—
Upload photos / approve / reject	—	—	—	✓
Set / revise price	✓	—	—	—
Accept / decline price	—	own	own	—
Mark earning paid	✓	—	—	—
View dashboard / KPIs	✓	—	—	—
View own monthly report	✓(all)	own	own	—

Note: the deliberate split — Admin prices/pays, Supervisor inspects, Worker accepts — creates a three-way separation of duties on the money path. No single role can invent, approve and pay for work alone.

6.3 User journeys

Admin

1. Log in on web → dashboard
2. Create project → add job cards with designs
3. Assign carpentry stage → (later) get 'approved, awaiting price' in the action queue
4. Set price → worker accepts → earning appears as Unpaid
5. Mark earning Paid → project completion rolls up

Worker (Carpenter/Painter)

6. Get assignment notification → open brief/designs
7. Do the work → tap 'Ready for Inspection'
8. Receive approval or rework notification
9. On approval + price: review → accept (done, earning created) or decline (Admin revises)
10. Check monthly report / export PDF

Supervisor

11. Get 'ready for inspection' notification
12. Open stage → inspect physically → capture photos
13. Approve (≥1 photo) → pricing unlocks, or Reject with reason → returns to worker

Phase 7 — API Planning

A versioned REST API (/api/v1) over HTTPS, JSON payloads, JWT bearer auth, RBAC-guarded on every route. Representative endpoints below; the full contract lives in the OpenAPI spec.

7.1 Conventions

- **Auth:** Authorization: Bearer <JWT>; access token ~15 min, refresh token rotated.
- **Errors:** consistent envelope { error: { code, message, details } } with correct HTTP status.
- **Validation:** request bodies validated (class-validator/DTO); 422 on validation failure.
- **Idempotency:** state-changing mobile actions accept an Idempotency-Key header.
- **Concurrency:** stage writes send If-Match / version; 409 on conflict.
- **Pagination:** cursor-based on list endpoints.

7.2 Representative endpoints

Method & path	Auth / role	Purpose
POST /api/v1/auth/login	public	Authenticate, return access+refresh tokens
POST /api/v1/auth/refresh	refresh token	Rotate access token
POST /api/v1/auth/password-reset	public	Email a verified reset link
POST /api/v1/users	Admin	Create user + assign role
PATCH /api/v1/users/{id}	Admin	Edit / deactivate user
POST /api/v1/projects	Admin	Create project
GET /api/v1/projects/{id}	any auth	Project detail + completion %
POST /api/v1/projects/{id}/job-cards	Admin	Create job card + attachments
POST /api/v1/stages/{id}/assign	Admin	Assign stage to role-matched worker
POST /api/v1/stages/{id}/ready	assignee	Mark Ready for Inspection
POST /api/v1/stages/{id}/photos	Supervisor	Upload inspection photo(s)
POST /api/v1/stages/{id}/approve	Supervisor	Approve (≥1 photo) → unlock pricing
POST /api/v1/stages/{id}/reject	Supervisor	Reject with reason → rework
POST /api/v1/stages/{id}/price	Admin	Set/revise price → Price Proposed
POST /api/v1/stages/{id}/price/accept	assignee	Accept → Completed + earning
POST /api/v1/stages/{id}/price/decline	assignee	Decline with reason
GET /api/v1/stages/{id}/pricing-history	involved parties	Full immutable history
POST /api/v1/earnings/{id}/pay	Admin	Mark Paid (date + admin)
GET /api/v1/dashboard	Admin	KPIs + action queues
GET /api/v1/reports/monthly?worker&month	self / Admin	Monthly earnings report
GET /api/v1/reports/monthly.pdf?...	self / Admin	PDF export
GET /api/v1/notifications	self	User notification feed

7.3 Example — set price (success & errors)

Request POST /api/v1/stages/{id}/price

```
{ "amount": 750000, "currency": "LKR" } // amount in minor units (cents)
```

200 OK

```
{ "stageId": "...", "status": "PRICE_PROPOSED", "amount": 750000, "version": 5 }
```

Status	Code	When
403	FORBIDDEN	Caller is not Admin
409	STAGE_NOT_APPROVED	Stage has not been Approved by a supervisor (BR-3)
409	VERSION_CONFLICT	Stale version — another write happened first
422	INVALID_AMOUNT	Amount ≤ 0 or above ceiling

Phase 8 — UI Planning

Screen-by-screen plan for the Admin web app and the mobile field app, with components, states and responsive behaviour. All screens follow the brand palette and WCAG 2.1 AA on key journeys.

8.1 Design system foundations

- **Palette:** forest green #4E7A54 (primary actions), brick red #B23A2E (accents/destructive & ‘needs attention’), cream #F7F4EE (background), dark #2C3A2E (text).
- **Every screen defines four states:** Loading (skeletons), Empty (guidance + primary CTA), Error (what went wrong + how to fix, per NFR-U.3), and Populated.
- **Touch targets ≥ platform minimum;** accept/decline reachable within 3 taps (NFR-U.1); dashboard KPIs visible without scrolling at 1366×768 (NFR-U.5).

8.2 Admin web screens

Screen	Key components	States / notes
Dashboard	KPI cards (active projects, stages by status, awaiting price, declined, unpaid total), action queues, charts	KPIs above the fold; queue rows deep-link to the stage
Projects list	Table, search, status filter, ‘New project’ button	Empty → ‘Create your first project’; archived hidden by default
Project detail	Job-card list, completion % ring, add-job-card dialog	Completion updates as stages approve
Job card detail	Stage tracker, assignee, status chips, attachment viewer, photos	Painting stage shown locked until carpentry approved
Pricing action queue	Rows of approved-awaiting-price & declined stages, inline set-price dialog	Primary daily Admin workflow
User management	User table, create/edit dialog, role selector, deactivate	Guarded: block deactivation with open work
Payments	Earnings table, Paid/Unpaid filter, mark-paid action	Confirmation on mark-paid

8.3 Mobile screens (field roles)

Screen	Key components	States / notes
Home / My tasks	Task cards by status, unread notification badge	Empty → ‘No assignments yet’
Stage detail (worker)	Design viewer, ‘Mark Ready’ button, status timeline	‘Mark Ready’ hidden unless In Progress
Price review (worker)	Proposed amount, Accept / Decline buttons, history link	Accept/Decline within 3 taps (NFR-U.1)
Inspection (supervisor)	Camera capture, photo grid, Approve / Reject (reason) buttons	Approve disabled until ≥1 photo; reject requires reason
Notifications	Feed, read/unread, deep links	Pull-to-refresh; badge clears on view
Monthly report	Month picker, jobs list, totals, ‘Download PDF’	Own data only

8.4 Navigation flow

- **Admin (web):** Login → Dashboard → {Projects, Pricing queue, Payments, Users}. The pricing action queue is the pinned daily entry point.
- **Mobile:** Login → Home/My tasks → Stage detail → (worker: Price review) / (supervisor: Inspection). Notifications are always one tap away.
- **Responsive:** Admin web is desktop-first but reflows to tablet; mobile app is portrait-first with large tap targets for workshop-floor use with gloves.

Phase 9 — Business Process

The end-to-end workflow, the stage state machine that governs it, decision points, notifications and escalations — current vs. future.

9.1 Current vs. future process

Dimension	Current (as-is)	Future (to-be)
Assignment	Verbal/phone	In-app, notified, recorded
Inspection	Subjective, no evidence	Supervisor photos + timestamped sign-off
Pricing	Ad hoc, unrecorded	Set after approval; every round in an immutable ledger
Acceptance	Implicit	Explicit worker accept/decline
Payment	Manual tally	Auto earning on accept; Admin marks paid
Visibility	None	Live dashboard + monthly reports

9.2 End-to-end workflow

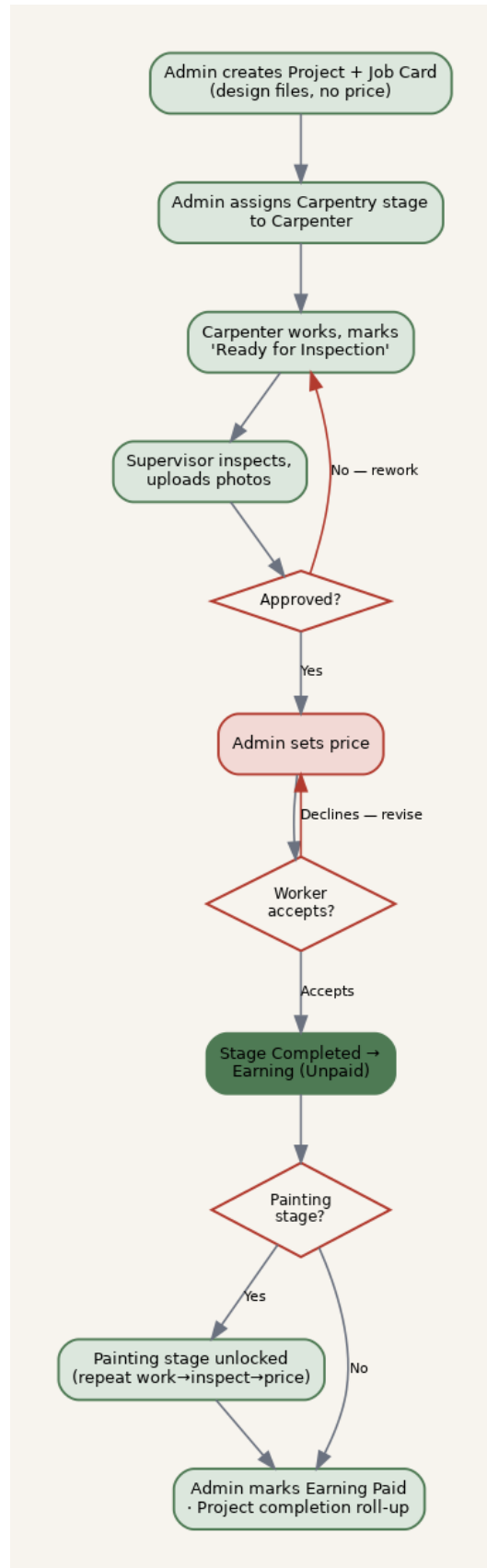


Figure 4 — End-to-end business process, from project creation to payment (green = happy path, red = money/decline branches).

9.3 Stage state machine

Every stage moves through exactly these nine states. Transitions are server-enforced; any move not shown is rejected. This is the single mechanism that guarantees BR-3, BR-5 and BR-6.

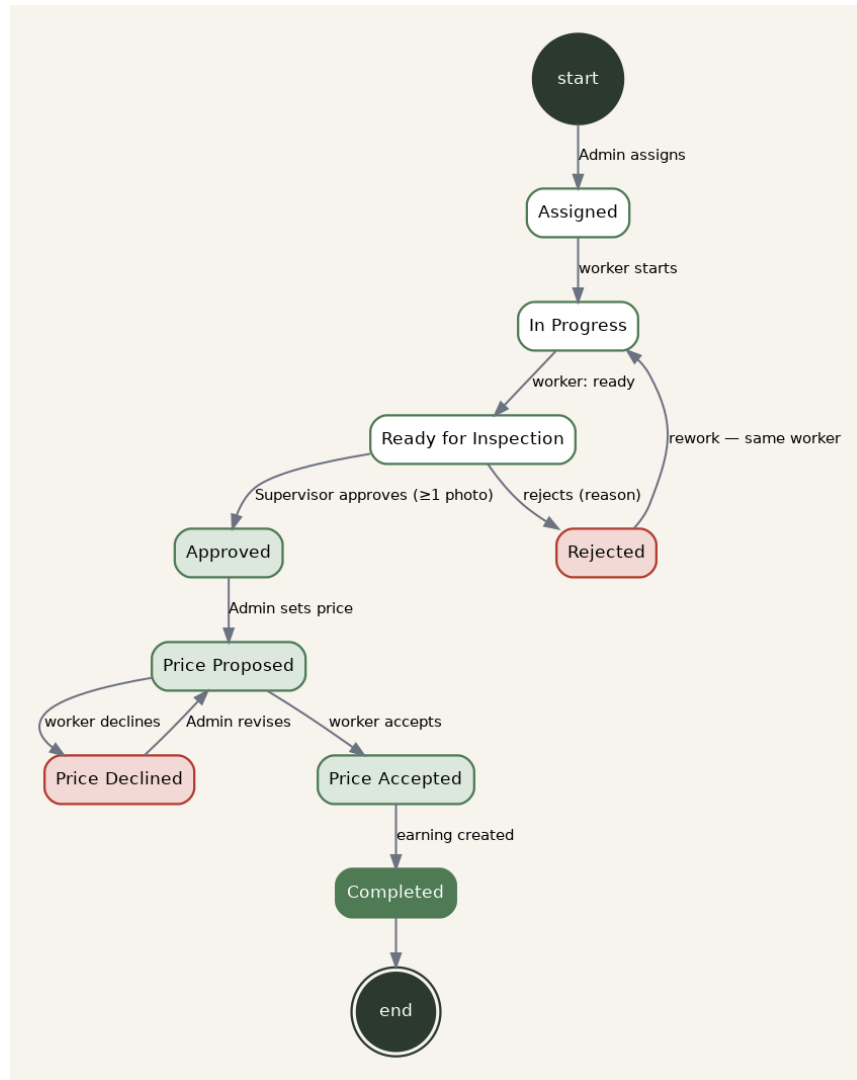


Figure 5 — Stage lifecycle state machine (nine states; guarded transitions).

9.4 Post-inspection pricing sequence

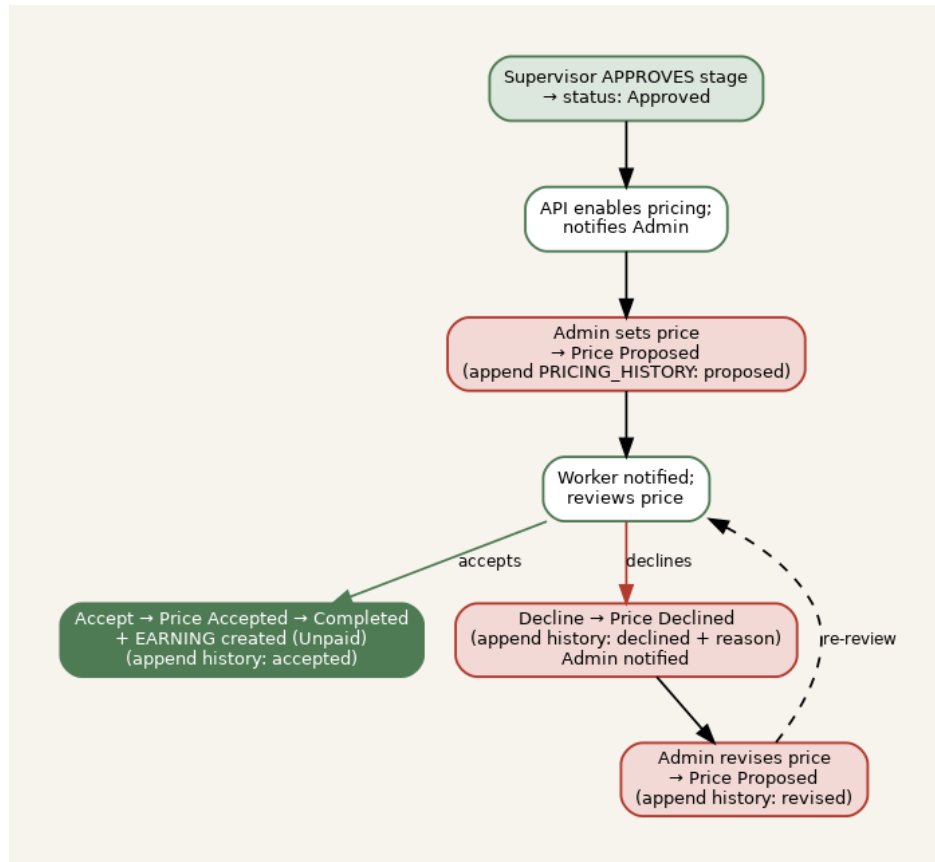


Figure 6 — Pricing sequence: approval → propose → accept/decline/review, each event appended to the ledger.

9.5 Decision points, notifications & escalations

Decision point	Outcomes	Notification / escalation
Supervisor inspection	Approve → pricing; Reject → rework	Worker notified either way; Admin queue updates on approve
Worker price review	Accept → Completed+earning; Decline → Admin revises	Admin notified on decline; worker notified on revised price
Painting gate	Locked until carpentry Approved	Painter assignment blocked until gate opens
Payment	Admin marks Paid	Worker's report reflects Paid status
Escalation (recommended, v2.0)	Stage stuck > N days at a state	Auto-alert Admin — closes the loop the SRS leaves open

Phase 10 — Security

Security is a first-order requirement here because the system determines what people are paid. The design aligns to OWASP ASVS and the SRS security constraints.

10.1 Authentication & session security

- **Passwords:** bcrypt, cost ≥ 12 (SRS constraint); checked against a breached-password list at set-time; never logged.
- **Tokens:** short-lived JWT access token (~15 min) + rotating refresh token; refresh reuse detection revokes the session family.
- **Reset:** single-use, time-boxed, verified-email reset links (FR-1.5).
- **Lockout:** progressive delay / lockout after repeated failures; every failure written to the audit log.

10.2 Authorization (RBAC)

Every endpoint is guarded by a role guard plus an ownership check (e.g. only the assigned worker acts on a stage; a worker sees only their own report). Authorisation is deny-by-default: a route with no explicit permission is inaccessible. Unauthorised attempts are blocked and recorded with user, timestamp and attempted action (FR-1.4).

10.3 OWASP Top-10 mitigations

Threat	Mitigation
Broken access control	Deny-by-default RBAC + ownership checks on every route; server-side enforcement only (never trust the client)
Injection (SQL)	Parameterised queries / ORM; no string-built SQL; least-privilege DB roles
XSS	Output encoding, React auto-escaping, strict Content-Security-Policy, sanitised rich text
CSRF	Token-based auth (no ambient cookies) for the API; SameSite + CSRF token if any cookie flow is used
Cryptographic failures	TLS 1.2+ in transit; AES-256 at rest (DB + object storage); secrets in a managed vault
SSRF / file abuse	Validate uploads by magic bytes; strip EXIF; private buckets; signed short-lived URLs
Security misconfiguration	Hardened container images, no debug in prod, dependency & image scanning in CI
Auth failures	bcrypt, lockout, rotating refresh tokens, MFA-ready
Logging/monitoring gaps	Central structured logs, audit log, alerting on anomalies
Vulnerable components	Automated SCA (dependency scanning) gate in CI; patch cadence

10.4 Rate limiting, secrets, audit

- Rate limiting at the gateway (per-IP and per-user) on auth and write endpoints to blunt brute-force and abuse.
- Secrets (DB creds, JWT keys, storage keys, push credentials) held in a managed secrets manager, never in code or images; rotated periodically.
- Audit log captures auth failures, permission denials, price/payment actions, and account changes — the backbone of dispute resolution.

- Media access always authorised by the API, then served via signed URLs expiring in minutes.

10.5 Compliance

Personal data (identities, payment records) supports correction and erasure where no legal retention rule applies. Financial records are retained per policy (TBD-04, recommend 7 years). Erasure requests run through a governed job that anonymises PII while preserving legally-required financial history — reconciling the erasure right with the retention duty.

Phase 11 — Performance & Scalability

Sizing, targets and the strategy to meet them. The system is small in data terms; the engineering effort goes into predictable latency and safe media handling, not raw scale.

11.1 Capacity estimate

Dimension	Estimate (order of magnitude)	Basis
Total users	Tens (4 roles, small workshop)	SRS user model
Concurrent users	200 target	NFR-P.3
Stages / month	Hundreds–low thousands	Job cards × 1–2 stages
Photos / month	~10× stages, ≤5 MB each	NFR-P.5
DB size / year	Low single-digit GB (rows); media dominates storage	Text rows are tiny; photos in object storage
Media storage / year	Tens–low hundreds of GB	Photo volume × retention

11.2 Performance targets & how we meet them

Target	Approach
Screens < 3s p95 (4G)	Indexed queries, cached dashboard aggregates (Redis, short TTL), CDN for static/media, lean payloads
Price action < 2s p95	Single-transaction write, no synchronous third-party calls on the path (notifications go async)
Photo upload < 15s p95	Direct-to-storage uploads via signed URLs, client-side compression, resumable/queued uploads
Report < 5s p95	Pre-aggregated monthly queries; PDF generated in a worker, streamed when ready
200 concurrent, <20% degradation	Stateless API scaled horizontally behind a load balancer; DB read replica for reports/dashboard
99.5% uptime	Multi-instance app, managed DB with failover, health checks, alerting, defined maintenance window

11.3 Caching, load balancing & CDN

- **Cache (Redis):** dashboard KPI aggregates, reference data, hot lookups; short TTL + event-based invalidation on state changes.
- **Load balancing:** gateway/LB distributes across stateless API instances; auto-scale on CPU/latency.
- **CDN:** serves the web SPA and, via signed URLs, media — keeping large files off the app tier and close to users.
- **DB optimisation:** the indexes in §4.4, read replica for reporting, connection pooling, and EXPLAIN-driven tuning on the few hot queries.

Phase 12 — DevOps

A right-sized delivery and operations setup: enough automation and safety for a money-handling system, without the overhead of infrastructure the team will not use.

12.1 Git & branching

- **Trunk-based with short-lived feature branches;** PRs require review + green CI + coverage gate before merge to main.
- **Protected main;** release tags drive deployments; conventional commits for automated changelogs.

12.2 Environments & CI/CD

Environment	Purpose
dev	Continuous integration; ephemeral/shared, seeded data
staging	Production-like; integration & performance testing
UAT	DesignArc acceptance sign-off with real user list
production	Single-tenant live system

CI/CD pipeline (GitHub Actions): lint → unit tests → build → SCA + image scan → integration tests → coverage gate (≥80% on business logic) → containerise → deploy to staging → (manual approval) → production. Database migrations run as a versioned, reversible step.

12.3 Containers, secrets, backups, monitoring

Concern	Approach	Why
Containers	Docker images; managed container runtime, Kubernetes-ready	Portable, reproducible; no premature k8s complexity
Secrets	Managed secrets manager; injected at runtime; rotated	Never in code/images (SRS security)
Backups	Automated daily DB backups + point-in-time recovery; object-storage versioning; periodic restore drills	‘Never lose pricing/photos/earnings’ (SRS criticality)
Monitoring	Metrics, health checks, uptime + error-rate alerts, structured logs	Meets 99.5% uptime NFR; feeds audit
Logging	Centralised, structured, correlation IDs; PII-scrubbed	Debuggability + compliance
DR	RPO ≤ 24h (daily) / minutes (PITR); documented restore runbook	Business-critical data protection

Phase 13 — Testing Strategy

Testing is organised around the SRS verification matrix (Test / Demonstration / Inspection / Analysis). The money path and the state machine get the deepest coverage.

Test type	Scope & focus	Tools / method
Unit	Business logic: state-machine transitions, pricing rules, RBAC guards, validators; ≥80% coverage on business modules	Jest (API), Flutter test, Vitest (web)
Integration	API + DB: transactional pricing→earning, sequence gate, append-only ledger, optimistic locking	Supertest against a real Postgres (containerised)
System / E2E	Full journeys: assign→ready→inspect→approve→price→accept→pay; rework and decline branches	Playwright (web), integration-driver (mobile)
UAT	DesignArc runs real scenarios against the requirement/acceptance criteria in UAT	Scripted acceptance cases from the RTM
Performance	p95 latency, 200-user load, photo-upload timing, report generation	Load tool (k6/JMeter); capacity analysis
Security	Auth/access-control tests, dependency & image scans, penetration test	OWASP ASVS checklist + pen-test
Regression	Automated suite re-run on every merge; critical-path smoke on deploy	CI pipeline
Accessibility	WCAG 2.1 AA on key journeys, contrast & touch-target scans	axe / platform a11y scanners + moderated usability

13.1 Automation strategy

- Unit + integration run on every PR (fast feedback); E2E on merge to main; performance & security on a schedule and pre-release.
- A ‘money-path’ test suite is tagged and always run — no release ships if pricing→earning tests fail.
- Test data seeded via factories; ephemeral DB per CI run for isolation.

Phase 14 — Documentation

The document set that surrounds a well-governed build. Several already exist (the SRS); the rest are produced across the programme.

Document	Purpose	Owner / status
Project Charter	Vision, scope, stakeholders, success criteria	PM — to produce
BRD	Business requirements & benefits	BA — partly in SRS
SRS	Functional & non-functional requirements	Exists (Rev 1.1)
SAD (this document)	Solution architecture & design decisions	Architecture — this deliverable
Database design	Schema, ERD, constraints, migration plan	Data — §4 herein, to expand
API documentation	OpenAPI spec + usage guide	Backend — living doc
RTM	Requirement → design → test traceability	QA — living doc
Test plan & test cases	Strategy + case library	QA
User guide	Admin & field-role how-tos	BA/UX
Admin guide	Account/role & operational admin	Ops
Deployment guide	Environment build & release steps	DevOps
Runbook	Incident, backup/restore, on-call procedures	Ops
Release notes	Per-version change summary	PM/Dev
Security & DPIA notes	Threat model, data-protection assessment	Security

Phase 15 — Project Planning

An epic → sprint plan aligned to the SRS phased roadmap (MVP → v1.0), with team, milestones, dependencies, effort and risks.

15.1 Epics & feature breakdown

Epic	Key features	Release
E1 Foundations	Repo, CI/CD, environments, auth, RBAC, user management	MVP
E2 Projects & Job Cards	Projects, job cards, stage model, sequence gate, attachments	MVP
E3 Assignment & Execution	Assign, mark ready, notifications (in-app)	MVP
E4 Inspection	Supervisor photos, approve/reject, evidence rule	MVP
E5 Pricing & Payments	Post-approval pricing, accept/decline/revise, ledger, earnings, mark paid	MVP
E6 Basic dashboard	Core KPIs, action queue	MVP
E7 Full dashboard & reports	Full KPIs, monthly PDF reports, workload views	v1.0
E8 Push & completion roll-up	Push notifications, project completion roll-up	v1.0
E9 Hardening	Performance, security/pen-test, accessibility, UAT fixes	v1.0

15.2 Indicative sprint plan (2-week sprints)

Sprint	Focus	Milestone
0	Discovery close-out, TBD resolution, environment & CI/CD setup, UX foundations	Kickoff / TBDs resolved
1–2	Auth, RBAC, user management, project & job-card CRUD	Walking skeleton
3–4	Stage model, assignment, execution, in-app notifications	Assign→ready flow
5–6	Inspection (photos, approve/reject), pricing, earnings, ledger	MVP money path E2E
7	Basic dashboard, MVP hardening, internal UAT	MVP release
8–9	Full dashboard, monthly PDF reports, push notifications, completion roll-up	v1.0 feature-complete
10–11	Performance, security & pen-test, accessibility, UAT, fixes	v1.0 go-live

Indicative timeline: ~5–6 months from kickoff to v1.0 go-live, MVP demonstrable around the end of month 3.

15.3 Team structure

Role	Count	Responsibility
Tech lead / architect	1	Architecture, code standards, hard problems (part shared)
Backend engineer	1–2	API, DB, state machine, pricing/payments
Mobile engineer (Flutter)	1	Field app
Web engineer (React)	1	Admin app
QA engineer	1	Test automation, RTM, UAT support
PM / BA (shared)	0.5	Backlog, stakeholders, UAT coordination

Role	Count	Responsibility
DevOps (shared)	0.5	Pipeline, environments, monitoring

15.4 Milestones, dependencies & effort

Milestone	Depends on	Effort (indicative)
TBDs resolved	DesignArc decisions (currency, retention, roles)	Sprint 0
Walking skeleton	Environments + auth	~2 sprints
MVP money path	Inspection + pricing + payments	~3 sprints
MVP release	Basic dashboard + hardening	End of month 3
v1.0 go-live	Reports, push, roll-up, security sign-off, UAT	End of month 5–6

15.5 Risk register (delivery)

Risk	P×I	Response
Unresolved TBDs delay start	Med×Med	Resolve in Sprint 0; defaults pre-agreed (Phase 1/2)
Workshop connectivity issues surface late	Med×Med	Test resumable uploads on real devices early
Scope creep to disbursement	Med×High	Change-control gate; hold record-only boundary
Single-supervisor bottleneck in ops	Med×Med	Multi-supervisor role model from MVP
Low-end device performance	Med×Med	Continuous testing on low-end Android

Phase 16 — Improvements Beyond the Requirements

Where DesignArc can go once the core is stable. Each item is mapped to the value it creates; none are needed for MVP.

16.1 AI, ML & automation

Idea	Value	Horizon
Suggested pricing from historical accepted prices per stage type/complexity	Faster, fairer, more consistent pricing; fewer decline loops	v2.0
Photo quality/anomaly check on inspection upload (blur/duplicate/AI-assisted defect hints)	Better evidence; fewer disputed approvals	v2.0+
Overdue/bottleneck detection & auto-escalation	Work never silently stalls	v2.0
Demand/throughput forecasting per worker & project	Better planning and deadlines	v3.0
Natural-language report/query for Admin	Self-serve insight without dashboards	v3.0

16.2 Analytics & enterprise features

- Rich analytics: cycle time per stage, rework rate per worker, price-negotiation patterns, project profitability.
- Separate Finance role and approval workflow (v2.0) — the design already anticipates this split.
- Multi-tenant capability if DesignArc scales to multiple sites/franchises.
- SSO + MFA for stronger account security as the org grows.
- Configurable stage templates (e.g. upholstery, finishing) so new work types need no code.

16.3 UX, notifications & integrations

- Offline-tolerant mobile capture (queue actions, sync when connected) — directly addresses the connectivity risk.
- WhatsApp/SMS fallback notifications for workers without reliable push.
- Client-facing portal (v3.0) for status transparency to DesignArc's own customers.
- Accounting & payroll integration (v3.0) once a package is confirmed — the natural path to actual disbursement.
- Materials/inventory module (v3.0) to connect the workflow to stock and true job costing.

16.4 Cost & security optimisations

- Lifecycle policies on object storage (move old photos to colder tiers) to cut media cost as history grows.
- Right-sized auto-scaling and scheduled scale-down outside working hours.
- Continuous dependency/image scanning and periodic pen-tests as standing practice, not one-off.
- Regular restore drills to prove — not assume — that the 'never lose data' guarantee holds.

Closing note. The DesignArc SRS is unusually disciplined, and the domain rewards that discipline: this is a system where being correct about money and evidence matters more than being clever about scale. The architecture above is deliberately conservative at the core — one relational source of truth, an explicit state machine, an immutable pricing ledger — and deliberately open at the edges, so that AI pricing, analytics, a Finance role, offline capture and eventual disbursement can all be added without disturbing the foundation. Build the boring, trustworthy core first; everything valuable later depends on it.

Immediate next step: resolve the five TBDs (we recommend LKR-only currency, 7-year financial retention, ≤10×5 MB photos, multi-supervisor role model, and Finance-role-ready permissions) so Sprint 0 can begin against a clean baseline.